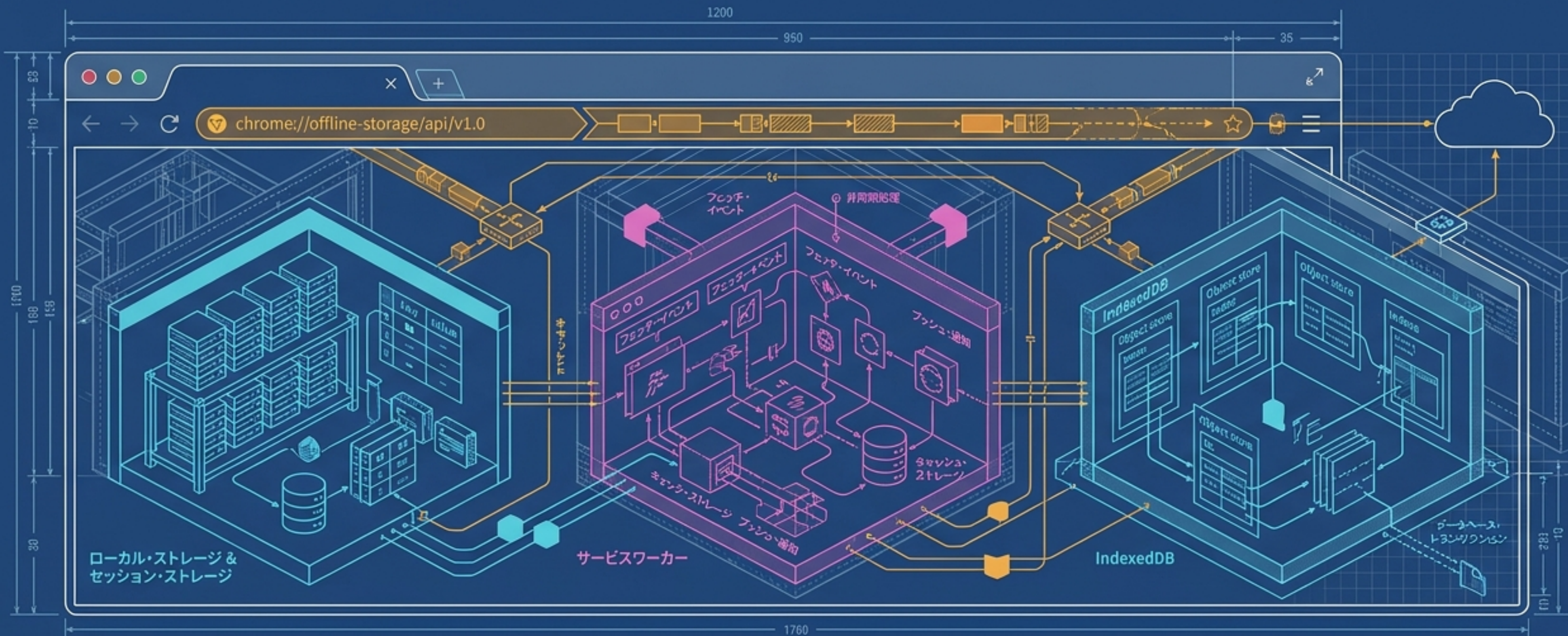


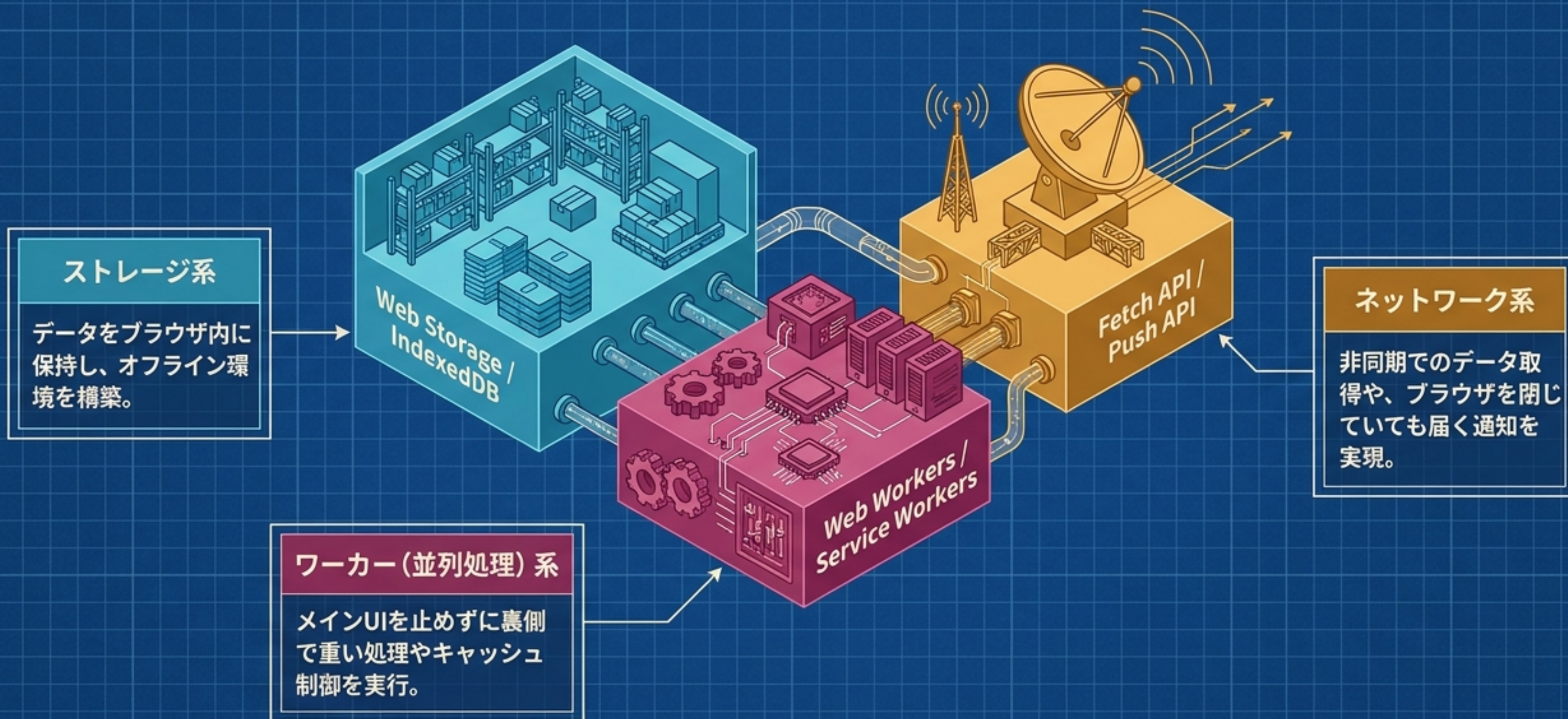
オフライン・ストレージAPI 完全攻略ブループリント

HTML5プロフェッショナル認定試験 レベル1 直前対策チートシート

🔥 頻出トラップ&コードスニペット完全網羅



ブラウザ拡張機能の全体見取り図



Storage Wars: Web Storage vs Cookie

🔥 試験の重要ポイント

	localStorage	sessionStorage	Cookie
保存期間	永続的	セッション中のみ (タブを閉じると消える)	有効期限を設定可能
スコープ	同一オリジン内で共有	同一タブ・同一オリジンのみ	ドメイン・パスで制御
容量	約5MB	約5MB	約4KB
サーバー送信	送信されない	送信されない	HTTPリクエストに自動付与
データ型	文字列のみ	文字列のみ	文字列のみ

⚠️ 注意

- ・オブジェクトの保存には `JSON.stringify()` が必須。
- ・storage イベントは「他のタブ・ウィンドウ」での変更のみ検知 (同一タブでは発火しない)。

大容量構造化データ：IndexedDB

🔥 試験の重要ポイント

項目	Web Storage	IndexedDB
データ型	文字列のみ	オブジェクト・バイナリも可
容量	約5MB	数百MB以上
操作方式	同期	非同期 (トランザクション)



⚠ 試験対策アラート

- すべての操作は **非同期**（結果は `onsuccess` / `onerror` で受け取る）。
- データ変更は必ず **トランザクション** 内（`readwrite` / `readonly`）で実行。
- スキーマ変更は **`onupgradeneeded`** 内でのみ可能。

UIをブロックしない並列処理：Web Workers

🔥 試験の重要ポイント

メインスレッド
(表舞台)

DOM操作・UI・イベント処理

待機中...

postMessage()

onmessage

Workerスレッド
(裏舞台)

重い処理（ループ計算など）をバックグラウンドで実行

⚠ 制約事項

- ・DOMへのアクセスは一切不可（document や window は使えない）。
- ・メインスレッドとのデータ受け渡しは参照ではなく **コピー（構造化クローン）** される。

Workerエコシステム：3つのWorkerの境界線

Dedicated Worker

特定のスクリプトに紐付いた専用Worker。
最も一般的。

```
new Worker('worker.js')
```

Shared Worker

複数のタブ・ウィンドウで共有できるWorker。

```
new SharedWorker()
```

port 経由で通信。

Service Worker

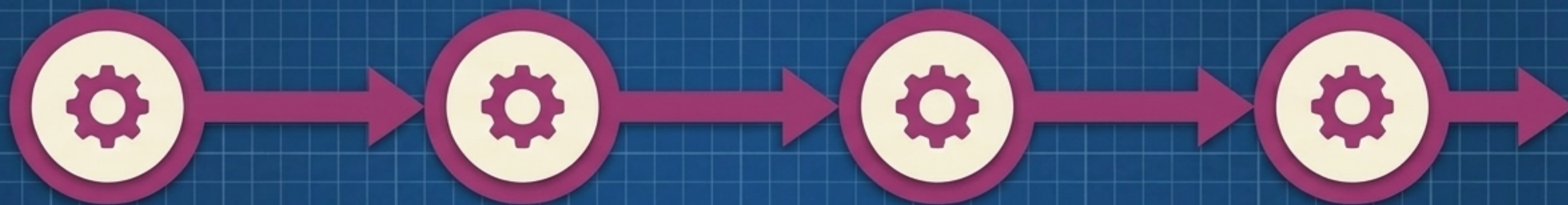
ブラウザとサーバーの間に立つ「プロキシ的」な特殊Worker。
オフライン対応・プッシュ通知の要。



すべてに共通する制約：DOMアクセス不可 / XMLHttpRequest・fetch・setTimeoutなどは利用可能

プロキシの核：Service Workerのライフサイクル

🔥 試験の重要ポイント



① register() (登録)

メインスレッドでスコープを定義して登録。

② install (インストール時)

静的リソース (HTML, CSS等) のキャッシュを実行。

③ activate (有効化時)

古いバージョンのキャッシュを削除・クリーンアップ。

④ fetch / push (待機・傍受)

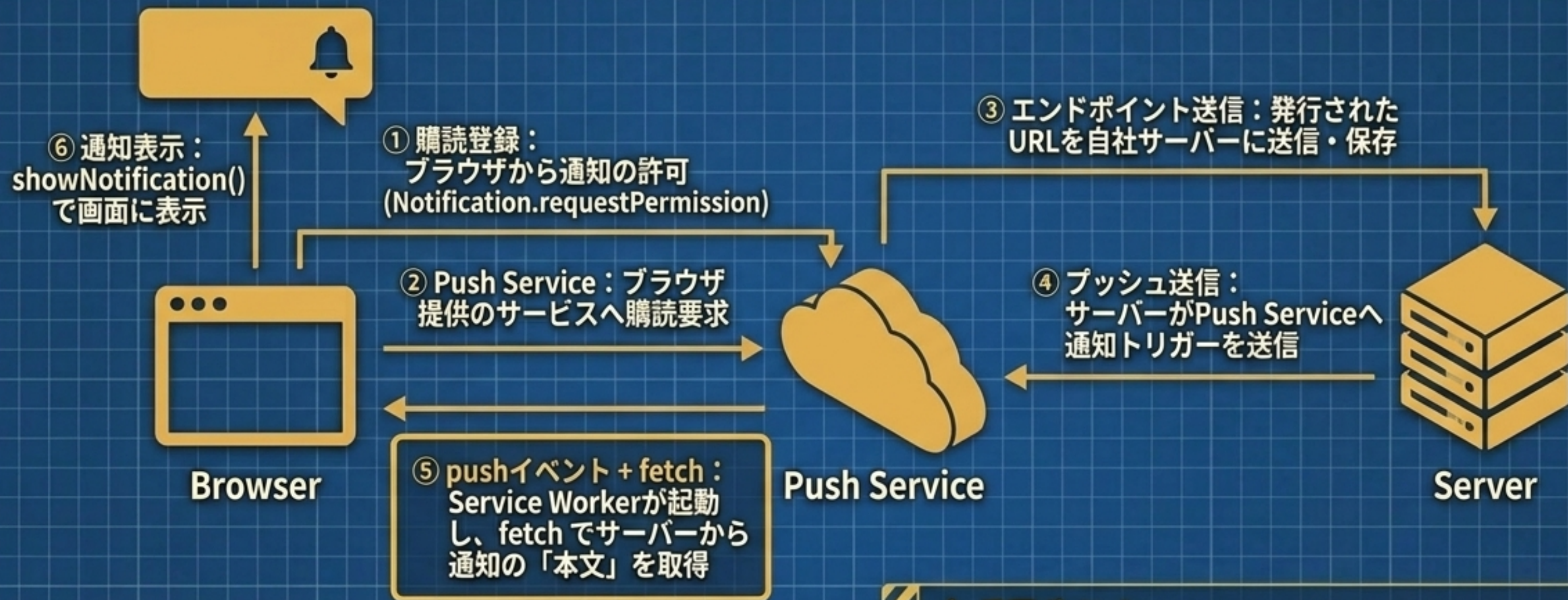
リクエストの傍受 (オフライン対応) やプッシュ通知の受信。

⚠ 必須要件

- **HTTPS環境必須** (ローカルテスト時の localhost は例外)。
- fetch イベントでネットワークリクエストを傍受し、**キャッシュから応答を返すことでオフライン対応を実現。**

ブラウザを閉じていても届く：Push APIの全体像

🔥 試験の重要ポイント



⚠ 重要ポイント

- VAPIDキーによるサーバー認証が必須。
- userVisibleOnly: true 必須 (サイレントプッシュ不可)。

モダンなHTTPリクエスト：Fetch API vs XHR

🔥 試験の重要ポイント

XMLHttpRequest (XHR)

```
xhr.open(...);  
xhr.onreadystatechange = function() {  
  if (xhr.readyState === 4 && xhr.status === 200) {  
    callback(...);  
  }  
};  
}
```

コールバックベース（ネストが深くなる）。
Service Workerでは利用不可。

Fetch API

```
async function getData() {  
  const response = await fetch(...);  
  if (response.ok) {  
    const data = await response.json(); // or .text()  
  } else {  
    // Handle errors  
  }  
}
```

Promiseベース（async/awaitでフラットに記述）。
Service Worker内でも利用可能。
レスポンスメソッド: `response.json()`, `response.text()` など。

⚠️ 試験トラップ警告

- ・ネットワークエラーは `.catch()` で捕捉できるが、404や500などのHTTPエラーは例外にならない！
- ・成否の判定は必ず `response.ok` (HTTPステータス 200～299) を確認すること。

必須コードスニペット・マッピング

これらのコア・メソッドのつづりと用途は、選択問題・穴埋め問題で確実に得点するための生命線です。

[Web Storage]	localStorage.setItem('name', 'Alice');
[Web Storage]	sessionStorage.getItem('token');
[IndexedDB]	db.transaction('users', 'readwrite');
[Web Worker]	worker.postMessage({ type: 'calc' });
[Service Worker]	navigator.serviceWorker.register('/sw.js');
[Fetch API]	await response.json();

⚠ 試験で狙われる「落とし穴」トラップ集

❌ 「タブを閉じるとデータが消えるのは localStorage である」

✅ 正解：それは sessionStorage。localStorageは永続的。Storage

❌ 「Web Storageはオブジェクトをそのまま保存できる」

✅ 正解：文字列のみ。JSON.stringify() が必要。

❌ 「Worker内から document オブジェクトでDOMを操作できる」

✅ 正解：WorkerはDOMアクセス 一切不可。

❌ 「Service WorkerはHTTP環境（本番）でも動作する」

✅ 正解：HTTPS必須（localhostは例外）。

❌ 「Fetch APIは、404エラー時に自動で .catch() に入る」

✅ 正解：入らない。response.ok で手動で判定する。Fetch API

The Master Blueprint: オフライン・ストレージAPI 総まとめ

API	概要	🔥 重要・引っかけポイント
localStorage	永続的KVS	文字列のみ（約5MB）。JSON変換必須。同一オリジン共有。
sessionStorage	セッションKVS	タブを閉じると消える。同一タブのみ。
IndexedDB	内蔵NoSQL DB	大容量・オブジェクト可・インデックス。非同期トランザクション。
Web Workers	バックグラウンドJS処理	UIブロック回避。DOM操作不可。postMessage通信。
Service Workers	ネットワークプロキシ	HTTPS必須。install → activate → fetch/push。
Push API	サーバーからの通知受信	pushイベント発火後、Fetch APIで本文取得。VAPID必須。
Fetch API	Promiseベース通信	Promiseを返す。SW内で利用可。response.okで成否確認。

この画面をスクリーンショットして試験直前に確認しましょう。Good Luck!